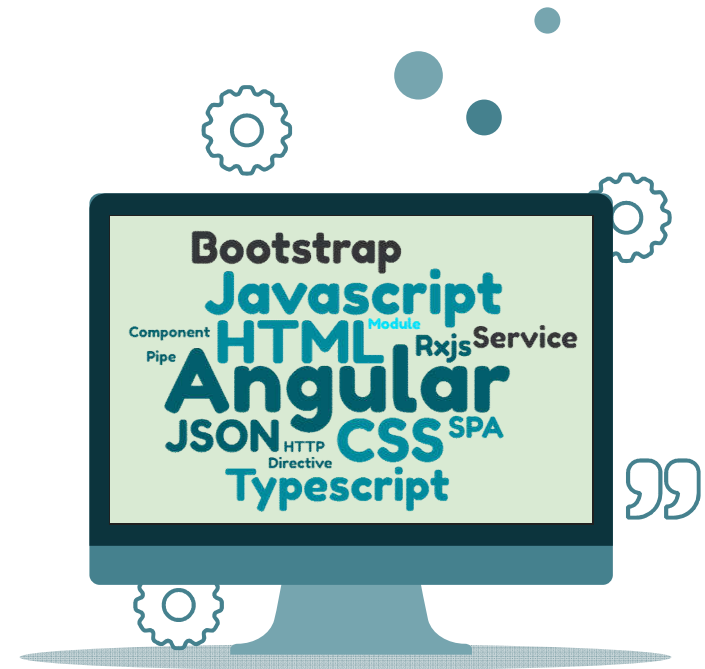


06

Formulaires





Chapitre 6 Formulaires

01

Template-driven forms

03

Atelier 4

02

Reactive forms



Que propose Angular ?

- Deux façons de créer les formulaires
- **Template-driven forms:**
 - Utilisant **FormsModule**
 - Facile et conseillé pour les formulaires simples
 - **Étapes** : Créer le formulaire dans le template, Angular l'analyse, puis vous met à disposition les différents inputs
- **Reactive Forms**
 - Basé sur **ReactiveFormsModule**
 - Façon complexe mais robuste, évolutif et conçu pour les applications nécessitant des contrôles particuliers (**FormGroup** et **FormBuilder**)
 - **Étapes** : Créer le formulaire dans le Template et dans le code TypeScript, puis lier les deux ensembles

01

Template-driven Forms



Template-driven forms : création

- Soit une interface personne qui servira de modèle

```
ng g i interfaces/personne
```

```
export interface Personne {  
  id? : number;  
  nom? : string;  
  prenom? : string;  
}
```

```
app > formulaire > ts formulaire.component.ts > ...  
import { Component, OnInit } from '@angular/core';  
import { Personne } from '../interfaces/personne';  
  
@Component({  
  selector: 'app-formulaire',  
  templateUrl: './formulaire.component.html',  
  styleUrls: ['./formulaire.component.css']  
})  
export class FormulaireComponent implements OnInit {  
  
  personne : Personne = {};  
  
  constructor() {}  
  
  ngOnInit() {}  
}
```

©MAD

```
p > formulaire > <> formulaire.component.html > ...  
<form>  
  <div class="form-group">  
    <!-- Pour utiliser ngModel dans un formulaire, il faut définir une valeur  
         pour l'attribut name de chaque élément du formulaire. -->  
    Nom : <input type="text" name="nom" [(ngModel)]="personne.nom">  
  </div>  
  <div class="form-group">  
    Prénom : <input type="text" name="prenom" [(ngModel)]="personne.prenom">  
  </div>  
  <div>  
    <button type="submit">  
      Ajouter  
    </button>  
  </div>  
</form>
```



Nom :

Prénom :

mohamedanouer.dahdeh@fsb.ucar.tn

Template-driven forms : Validation

- Utiliser la directive `ngForm` pour créer une variable locale (référence) associée au formulaire

```
<form #monForm=ngForm>
  <div class="form-group">
    <!-- Pour utiliser ngModel dans un formulaire, il faut définir une valeur
         pour l'attribut name de chaque élément du formulaire. -->
    Nom : <input type="text" name="nom" [(ngModel)]="personne.nom" required>
  </div>
  <div class="form-group">
    Prénom : <input type="text" name="prenom" [(ngModel)]="personne.prenom" required>
  </div>
  <div>
    <button type="submit" [disabled]="!monForm.valid">
      Ajouter
    </button>
  </div>
</form>
```



Nom :

Prénom :

Ajouter

Template-driven forms : Validation

```
<form #monForm=ngForm>
  <div class="form-group">
    Nom : <input type="text" name="nom" [(ngModel)]="personne.nom" required #nom="ngModel">
  </div>
  <div [hidden]="nom.valid || nom.pristine"> ←
    Le nom est obligatoire
  </div>
  <div class="form-group">
    Prénom : <input type="text" name="prenom" [(ngModel)]="personne.prenom" required #prenom="ngModel">
  </div>
  <div [hidden]="prenom.valid || prenom.pristine">
    Le prénom est obligatoire
  </div>
  <div>
    <button type="submit" [disabled]="!monForm.valid">
      Ajouter
    </button>
  </div>
</form>
```

```
app > formulaire > # formulaire.component.css > ...
.ng-invalid:not(form){
  border: 1px solid red;
}
.ng-valid:not(form){
  border: 1px solid green;
}
```

Nom :

Prénom :

Ajouter

Nom :

Prénom :

Ajouter

Nom :

Prénom :

Ajouter

Nom :

Prénom :

Le prénom est obligatoire
Ajouter

Template-driven forms : Explication

- `ng-untouched` : classe css Angular appliquée quand le champ n'est pas touché (son inverse est `ng-touched`)
- `ng-pristine` : classe css Angular appliquée quand le champ est vide (son inverse est `ng-dirty`)
- `ng-invalid` : classe css Angular appliquée quand le champ n'est pas valid (son inverse est `ng-valid`)

Remarque

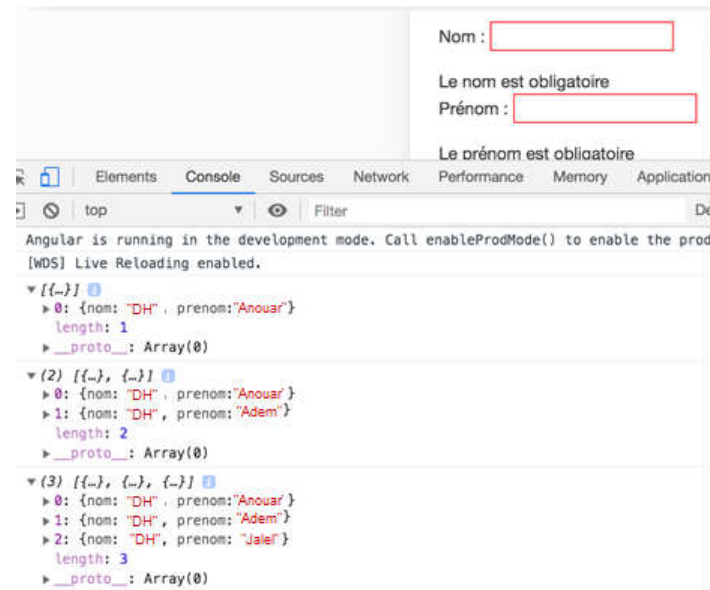
La pseudo-classe de négation, `:not()`, est une notation fonctionnelle qui prend un sélecteur comme argument. Elle permet de cibler les éléments qui ne sont pas représentés par cet argument.

Template-driven forms : Soumission

- Pour soumettre un formulaire, on utilise la directive **ngSubmit**

```
<form #monForm=ngForm (ngSubmit) = ajouterPersonne(>
```

```
export class FormulaireComponent implements OnInit {  
  personnes: Array<Personne> = [];  
  personne : Personne = {};  
  
  constructor() { }  
  
  ngOnInit() { }  
  
  ajouterPersonne() {  
    this.personnes.push({ ...this.personne });  
    this.personne.nom = '';  
    this.personne.prenom = '';  
    console.log(this.personnes);  
  }  
}
```



Exercice

#8

Entant que utilisateur authentifié
je veux ajouter un nouveau
contact

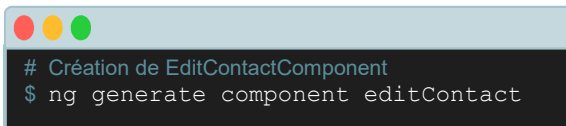
The screenshot shows a web browser at localhost:4200/contacts/edit. The application has a blue header with the text 'first-project' and navigation links: Home, About, Contacts, and Signin. Below the header is a blue bar labeled 'Edit Contact Component'. The main form is titled 'Add Contact' and contains the following fields:

- Name:** A text input field containing 'Sergey Brin'.
- Email:** A text input field containing 'sergey.brin@gmail.com'.
- Website:** A text input field containing 'www.google.com'.
- Projects (separated with comma","):** A text input field containing 'Google,Alphabet Inc.'.

At the bottom of the form is a blue 'Save' button. Below the form is a blue button labeled 'Go to Contacts'.

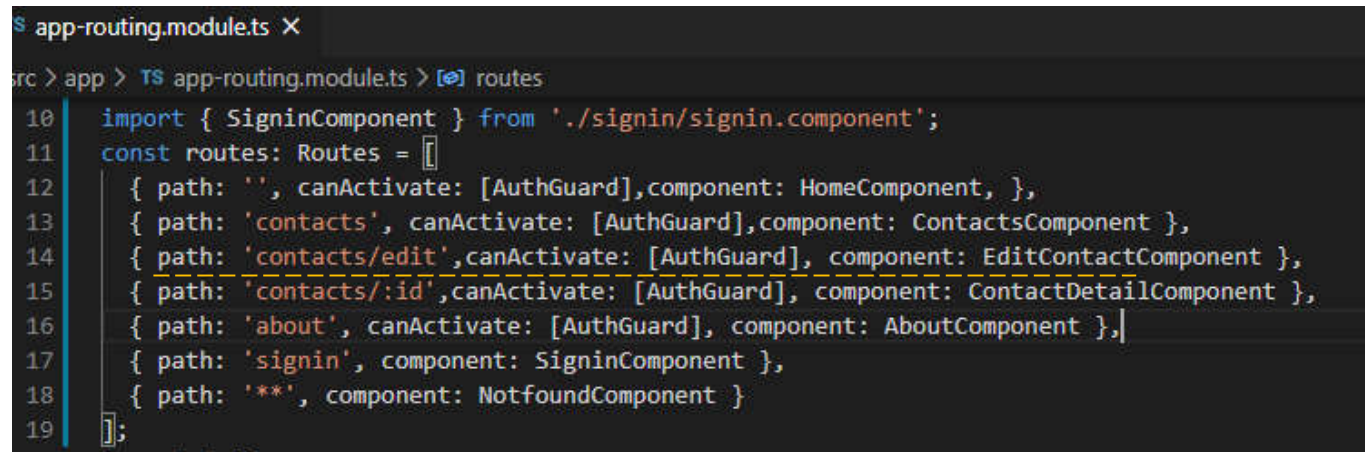
Formulaire : Ajouter un contact

- **Etape 1:** créer le component **EditContactComponent** :



```
# Création de EditContactComponent
$ ng generate component editContact
```

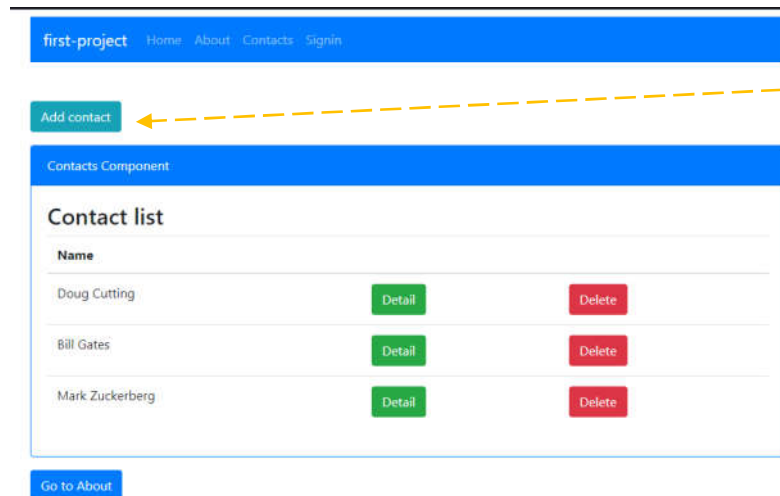
- **Etape 2:** ajouter la route au **routing-module**



```
$ app-routing.module.ts X
src > app > TS app-routing.module.ts > routes
10 import { SigninComponent } from './signin/signin.component';
11 const routes: Routes = [
12   { path: '', canActivate: [AuthGuard], component: HomeComponent, },
13   { path: 'contacts', canActivate: [AuthGuard], component: ContactsComponent },
14   { path: 'contacts/edit', canActivate: [AuthGuard], component: EditContactComponent },
15   { path: 'contacts/:id', canActivate: [AuthGuard], component: ContactDetailComponent },
16   { path: 'about', canActivate: [AuthGuard], component: AboutComponent },
17   { path: 'signin', component: SigninComponent },
18   { path: '**', component: NotfoundComponent }
19 ];
```

Formulaire : Ajouter un contact

- **Etape 3:** ajouter le bouton Add Contact à ContactsComponent



```
contacts.component.html X
src > app > contacts > contacts.component.html > div.card.border-primary.mb-3.mt-3
1 <button class="btn btn-info" (click)="onAddContact()">
2   Add contact
3 </button>
4 <div class="card border-primary mb-3 mt-3">
5   <div class="card-header bg-primary text-white"> Contacts Component</div>
```

```
TS contacts.component.ts X
src > app > contacts > TS contacts.component.ts > ...
22   }
23   onAddContact() {
24     this.router.navigate(['/contacts/edit']);
25   }
26 }
```

Formulaire : Ajouter un contact

- Etape 4: préparer le Template (edit-contact.component.html)

```
edit-contact.component.html X
src > app > edit-contact > edit-contact.component.html > div.card.border-primary.mb-3 > div.card-body > form > button.btn.btn-primary
1 <div class="card border-primary mb-3">
2   <div class="card-header bg-primary text-white"> Edit Contact Component</div>
3   <div class="card-body">
4     <h3>Add Contact</h3>
5     <form (ngSubmit)="onSubmit(form)" #form="ngForm"> <!-- #form référence locale-->
6       <div class="form-group">
7         <label for="name">Name</label>
8         <input type="text" id="name" class="form-control" name="name" ngModel >
9       </div>
10      <div class="form-row">
11        <div class="col">
12          <label for="email">Email</label>
13          <input type="email" id="email" class="form-control" name="email" ngModel>
14        </div>
15        <div class="col">
16          <label for="website">Website</label>
17          <input type="url" id="website" class="form-control" name="website" ngModel>
18        </div>
19      </div>
20      <div class="form-group">
21        <label for="projects">Projects (separated with comma",")</label>
22        <input type="text" id="projects" class="form-control" name="projects" ngModel>
23      </div>
24      <button type="submit" class="btn btn-primary"> Save</button>
25    </form>
26  </div>
27 </div>
28 <button class="btn btn-primary" (click)="onContacts()">
29   Go to Contacts
30 </button>
```

<label> peut être associé à un contrôle en utilisant l'attribut for.

Edit Contact Component

Add Contact

Name
Sergey Brin

Email
sergey.brin@gmail.com

Website
www.google.com

Projects (separated with comma",")
Google,Alphabet Inc.

Save

Formulaire : Ajouter un contact

- **Etape 5:** Traitement associé à la soumission du formulaire (edit-contact.component.ts):

```
TS edit-contact.component.ts X
src > app > edit-contact > TS edit-contact.component.ts > ...
3 import { Router } from '@angular/router';
4
5 @Component({
6   selector: 'app-edit-contact',
7   templateUrl: './edit-contact.component.html',
8   styleUrls: ['./edit-contact.component.css']
9 })
10 export class EditContactComponent implements OnInit {
11
12   constructor(private router: Router) { }
13
14   ngOnInit(): void {
15   }
16   onSubmit(form: NgForm){
17     //it's provisional
18     console.log(form.value);
19   }
20   onContacts() {
21     this.router.navigate(['/contacts']);
22   }
23 }
```

The screenshot shows a web browser at `localhost:4200/contacts/edit` displaying the 'Edit Contact Component' page. The page has a blue header with the title 'Add Contact'. Below the header, there are four input fields: 'Name' (containing 'Sergey Brin'), 'Email' (containing 'sergey.brin@gmail.com'), 'Website' (containing 'www.google.com'), and 'Projects (separated with comma,")' (containing 'Google, Alphabet Inc'). A blue 'Save' button is located below the 'Projects' field. At the bottom of the form, there is a blue 'Go to Contacts' button.

The Chrome DevTools console is open, showing a log message from `edit-contact.component.ts:18` with the following object:

```
{name: "Sergey Brin", email: "sergey.brin@gmail.com", website: "www.google.com", projects: "Google, Alphabet Inc."}
```

Exercice (Contrôle de saisie)

#4 BFN

Règles de validation :

- ✓ Les champs sont obligatoires
- ✓ Une adresse e-mail valide
- ✓ Une URL valide

Edit Contact Component

Add Contact

Name

Please provide a name.

Email

Please provide a valid email.

Website

Please provide a valid URL (http://...).

Projects (separated with comma ",")

Please provide a project.

Save

Validation : Formulaire

- **Etape 1:** Ajout des règles de validation au Template (edit-contact.component.html):
- **Etape 2 :** Désactivation du bouton **save** si le formulaire est **invalide**

```
edit-contact.component.html X
src > app > edit-contact > edit-contact.component.html > ...
1 <div class="card border-primary mb-3">
2   <div class="card-header bg-primary text-white"> Edit Contact Component</div>
3   <div class="card-body">
4     <h3>Add Contact</h3>
5     <form (ngSubmit)="onSubmit(form)" #form="ngForm"> <!-- #form référence locale -->
6       <div class="form-group">
7         <label for="name">Name</label>
8         <input type="text" id="name" class="form-control" name="name" required ngModel >
9       </div>
10      <div class="form-row">
11        <div class="col">
12          <label for="email">Email</label>
13          <input type="email" id="email" class="form-control" name="email"
14            required pattern="[a-z0-9._%+-]+@[a-z0-9.-]+\.[a-z]{2,4}$" ngModel>
15        </div>
16        <div class="col">
17          <label for="website">Website</label>
18          <input type="url" id="website" class="form-control" name="website"
19            required pattern="https://.+>
20          </div>
21        </div>
22        <div class="form-group">
23          <label for="projects">Projects (separated with comma",")</label>
24          <input type="text" id="projects" class="form-control" name="projects" required ngModel>
25        </div>
26        <button type="submit" class="btn btn-primary" [disabled]="form.invalid">Save</button>
27      </form>
28    </div>
29  </div>
30  <button class="btn btn-primary" (click)="onContacts()">
31    Go to Contacts
32  </button>
33
```

Validation : Formulaire

- Etape 3 : Ajout des messages d'erreur

```
4 <h3>Add Contact</h3>
5 <form (ngSubmit)="onSubmit(form)" #form="ngForm"> <!-- #form référence locale-->
6   <div class="form-group">
7     <label for="name">Name</label>
8     <input type="text" id="name" class="form-control" name="name" [ngClass]="{'is-invalid':name.invalid && name.dirty}"
9       #name="ngModel" required ngModel >
10    <div *ngIf="name.invalid && name.dirty" class="invalid-feedback">Please provide a name.</div>
11  </div>
12  <div class="form-row">
13    <div class="col">
14      <label for="email">Email</label>
15      <input type="email" id="email" class="form-control" [ngClass]="{'is-invalid':email.invalid && email.dirty}"
16        name="email" #email="ngModel" required pattern="[a-z0-9._%+-]+@[a-z0-9.-]+\.[a-z]{2,4}$" ngModel>
17      <div *ngIf="email.invalid && email.dirty" class="invalid-feedback">Please provide a valid email address. </div>
18    </div>
19    <div class="col">
20      <label for="website">Website</label>
21      <input type="url" id="website" class="form-control" [ngClass]="{'is-invalid':website.invalid && website.dirty}"
22        name="website" #website="ngModel" required pattern="https?://.+>
23      <div *ngIf="website.invalid && website.dirty" class="invalid-feedback"> Please provide a valid URL (http://...)</div>
24    </div>
25  </div>
26  <div class="form-group mt-2">
27    <label for="projects">Projects (separated with comma",")</label>
28    <input type="text" id="projects" class="form-control" name="projects" [ngClass]="{'is-invalid':projects.invalid && project.dirty}"
29      #projects="ngModel" required ngModel >
30    <div *ngIf="projects.invalid && projects.dirty" class="invalid-feedback">Please provide a project.</div>
31  </div>
32  <button type="submit" class="btn btn-primary" [disabled]="form.invalid">Save</button>
33 </form>
```

The diagram illustrates the mapping between the HTML code and the rendered form. A yellow arrow points from the error message in the code to the error message in the form. Another yellow arrow points from the input field in the code to the input field in the form.

sb.ucar.tn

Résultat

Edit Contact Component

Add Contact

Name

Email Website

Projects (separated with comma,"")

Save

Edit Contact Component

Add Contact

Name

Email Website

Please provide a valid email address. Please provide a valid URL (http://...).

Projects (separated with comma,"")

Please provide a project.

Save

Edit Contact Component

Add Contact

Name

Please provide a name.

Email Website

Please provide a valid email address. Please provide a valid URL (http://...).

Projects (separated with comma,"")

Please provide a project.

Save

Edit Contact Component

Add Contact

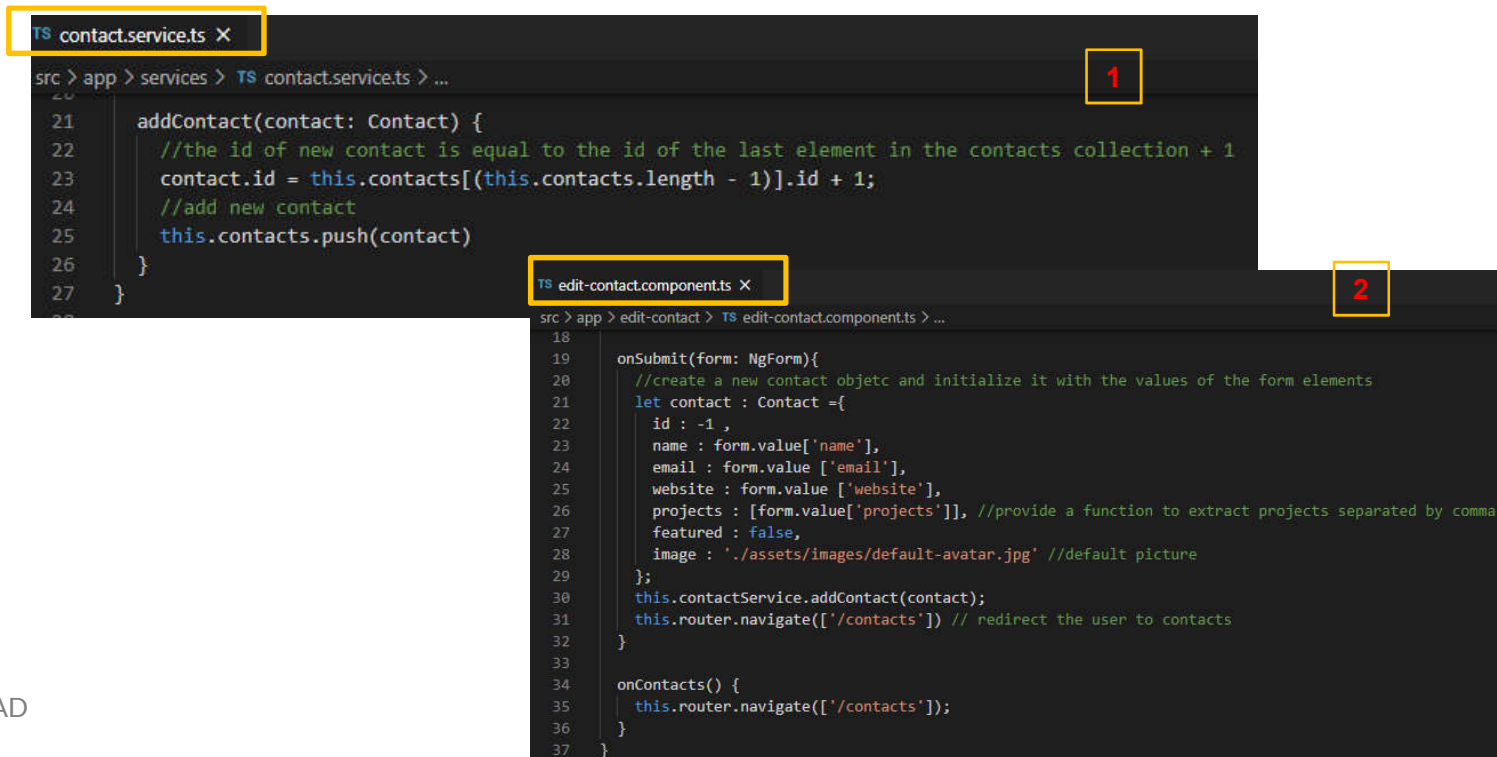
Name

Email Website

Projects (separated with comma,"")

Save

Terminer le traitement dans le **ContactService** et la classe du composant **EditContactComponent**

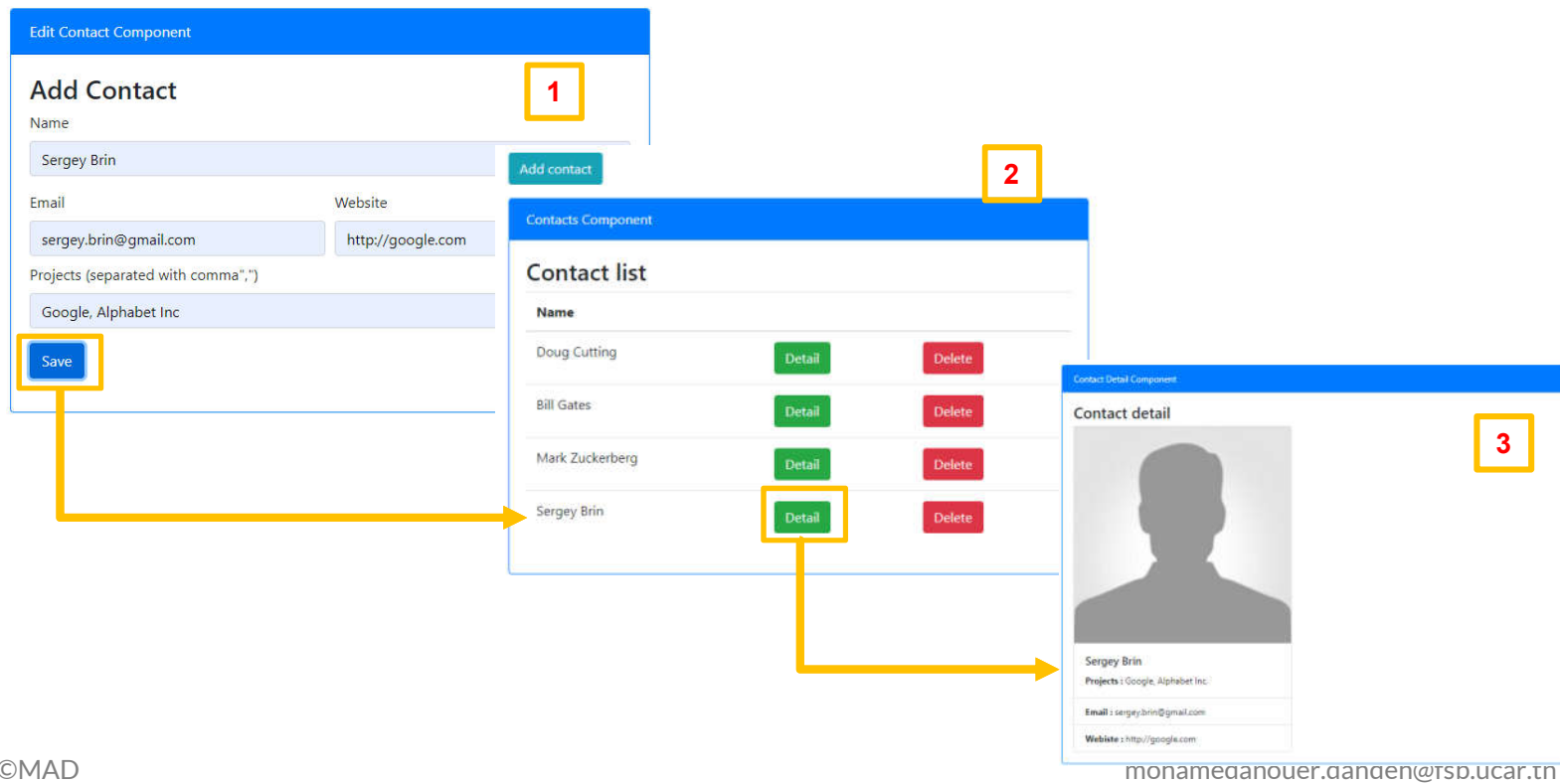


The image shows two overlapping code editor windows. The top window, titled 'TS contact.service.ts', contains the implementation of the `addContact` method in the `ContactService` class. The bottom window, titled 'TS edit-contact.component.ts', contains the implementation of the `onSubmit` and `onContacts` methods in the `EditContactComponent` class. Both windows have yellow boxes highlighting their titles and red boxes with numbers 1 and 2 indicating specific parts of the code.

```
src > app > services > TS contact.service.ts > ...  
21 addContact(contact: Contact) {  
22     //the id of new contact is equal to the id of the last element in the contacts collection + 1  
23     contact.id = this.contacts[(this.contacts.length - 1)].id + 1;  
24     //add new contact  
25     this.contacts.push(contact)  
26 }  
27 }
```

```
src > app > edit-contact > TS edit-contact.component.ts > ...  
18  
19 onSubmit(form: NgForm){  
20     //create a new contact objetc and initialize it with the values of the form elements  
21     let contact : Contact = {  
22         id : -1 ,  
23         name : form.value['name'],  
24         email : form.value ['email'],  
25         website : form.value ['website'],  
26         projects : [form.value['projects']], //provide a function to extract projects separated by comma  
27         featured : false,  
28         image : './assets/images/default-avatar.jpg' //default picture  
29     };  
30     this.contactService.addContact(contact);  
31     this.router.navigate(['/contacts']) // redirect the user to contacts  
32 }  
33  
34 onContacts() {  
35     this.router.navigate(['/contacts']);  
36 }  
37 }
```

Résultat final



02

Reactive Forms



Reactive Form

- Pour l'active vous devrez l'importer dans `app.module.ts`

```
import { FormsModule, ReactiveFormsModule } from '@angular/forms';
```

```
imports: [  
  BrowserModule,  
  FormsModule,  
  AppRoutingModule,  
  ReactiveFormsModule  
],
```

- **FormControl**

- Une classe Angular
- Permettant d'associer un attribut de composant à un champ du formulaire défini dans le Template associé afin de faciliter
 - le binding
 - le contrôle et la validation

- **FormGroup**

- Une classe Angular
- Composée de plusieurs objets de type **FormControl**

Reactive Form : FormGroup & FormControl

- **Etape 1** : Définir un objet de type FormGroup contenant des propriétés (attributs) de type FormControl
- **Etape 2** : Créer le formulaire

Identifiant :

Nom :

Prénom :

▼ Object 
id: 1
nom: "Dahdeh"
prenom: "Mohamed Anouar"
► __proto__: Object

```
app > reactive-form > ts reactive-form.component.ts > ...
import { Component, OnInit } from '@angular/core';
import { FormControl, FormGroup } from '@angular/forms';

@Component({
  selector: 'app-reactive-form',
  templateUrl: './reactive-form.component.html',
  styleUrls: ['./reactive-form.component.css']
})

export class ReactiveFormComponent implements OnInit {

  personneForm = new FormGroup({
    id: new FormControl(''),
    nom: new FormControl(''),
    prenom: new FormControl('')
  });

  constructor() {}

  ngOnInit() {}

  afficherTout() {
    console.log(this.personneForm.value);
  }
}
```

```
app > reactive-form > <> reactive-form.component.html > ...
<form [formGroup]="personneForm">
  <div class="form-group">
    Identifiant :
    <input type="number" formControlName="id">
  </div>
  <div class="form-group">
    Nom :
    <input type="text" formControlName="nom">
  </div>
  <div class="form-group">
    Prénom :
    <input type="text" formControlName="prenom">
  </div>
  <button (click)="afficherTout()">cliquer</button>
</form>
```

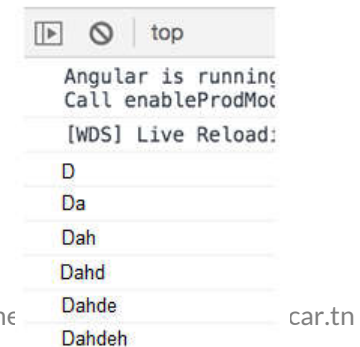
mohamedanouer.dahdeh@fsb.ucar.tn

Reactive Form : FormGroup & FormControl

- Pour récupérer le `FormControl` associé à `nom`
`this.personneForm.controls.nom` ou `this.personneForm.get('nom')`
- Pour vider les champs d'un formulaire : `this.personneForm.reset();`
- Initialiser les champs :
`this.personneForm.setValue({nom: 'Tounsi', prenom: 'Adam', id: 1});`
- Pour surveiller le changement de valeur d'un champ: `valueChanges`

```
ngOnInit() {  
  this.personneForm.controls.nom.valueChanges.subscribe(change => {  
    console.log(change);  
  });  
}
```

Nom :



Reactive Form : Soumission

- Ajouter un event binding (`ngSubmit`) avec un bouton de type `submit`

```
<form [formGroup]="personneForm" (ngSubmit) = "afficherTout()">
  <div class="form-group">
    Identifiant :
    <input type="number" formControlName="id">
  </div>
  <div class="form-group">
    Nom :
    <input type="text" formControlName="nom">
  </div>
  <div class="form-group">
    Prénom :
    <input type="text" formControlName="prenom">
  </div>
  <button type="submit">Envoyer</button>
</form>
```

Reactive Form : Validation

- La classe `FormControl` peut prendre deux paramètres :
 - la `valeur initiale` à afficher dans le formulaire
 - une ou plusieurs `règles de validation`
- Pour définir une règle de validation, on peut utiliser la classe `Angular Validators` contenant plusieurs règles de validation

```
personneForm = new FormGroup({  
  id: new FormControl('', Validators.required),  
  nom: new FormControl('', [Validators.pattern(/^[A-Z][a-z]{2,10}/), Validators.required] ),  
  prenom: new FormControl('', [ Validators.required,  
    this.checkPrenomValidator ])  
});
```



```
checkPrenomValidator(control: FormControl)  
{  
  const str: string = control.value;  
  if (str[0] >= 'A' && str[0] <= 'Z') {  
    return null;  
  } else {  
    return {erreur: 'Prénom non valide'};  
  }  
}
```

Reactive Form : Message d'erreur

```
<form [formGroup]="personneForm" (ngSubmit) = "afficherTout()">
  <div class="form-group">
    Identifiant :
    <input type="number" formControlName="id">
  </div>
  <div *ngIf="id.invalid && (id.dirty || id.touched)">
    <div *ngIf="id.errors?.required">L'identifiant est obligatoire</div>
  </div>
  <div class="form-group">
    Nom :
    <input type="text" formControlName="nom">
  </div>
  <div *ngIf="nom.invalid && (nom.dirty || nom.touched)">
    <div *ngIf="nom.errors?.required">Le nom est obligatoire</div>
    <div *ngIf="nom.errors?.pattern">1ère lettre en majuscule et min 3 lettres max 11</div>
  </div>
  <div class="form-group">
    Prénom :
    <input type="text" formControlName="prenom">
  </div>
  <div *ngIf="prenom.invalid && (prenom.dirty || prenom.touched)">
    <div *ngIf="prenom.errors?.required">Le prénom est obligatoire</div>
    <div *ngIf="prenom.errors?.erreur">1ère lettre en majuscule</div>
  </div>
  <button type="submit" [disabled]='!personneForm.valid'>Envoyer</button>
</form>
```

Reactive Form : Message d'erreur

- On ne peut accéder à la propriété `invalid` ni `errors` si on ne définit pas de `getter` pour chaque `FormControl` dans le composant

```
Angular is running in the development mode. core.js:38781
Call enableProdMode() to enable the production mode.

[WDS] Live Reloading enabled. client:52

✖ ERROR TypeError: Cannot read ReactiveFormComponent.html:5
property 'invalid' of undefined
    at Object.eval [as updateDirectives] (ReactiveFormComponent.html:7)
    at Object.debugUpdateDirectives [as updateDirectives]
    (core.js:45259)
    at checkAndUpdateView (core.js:44271)
    at callViewAction (core.js:44637)
    at execComponentViewsAction (core.js:44565)
    at checkAndUpdateView (core.js:44278)
    at callViewAction (core.js:44637)
    at execEmbeddedViewsAction (core.js:44594)
    at checkAndUpdateView (core.js:44272)
    at callViewAction (core.js:44637)

✖ ERROR CONTEXT ReactiveFormComponent.html:5
  DebugContext_ {view: {...}, nodeIndex: 11, nodeDef: {...}, el
  Def: {...}, elView: {...}}
```

```
get nom() {
  return this.personneForm.get('nom');
}

get id() {
  return this.personneForm.get('id');
}

get prenom() {
  return this.personneForm.get('prenom');
}
```

Identifiant :

L'identifiant est obligatoire

Nom :

1ère lettre en majuscule et min 3 lettres max 11

Prénom :

1ère lettre en majuscule

Reactive Form : FormGroup Imbriqué

- Définition des FormGroup imbriqués



```
personneForm = new FormGroup({
  id: new FormControl('', Validators.required),
  nom: new FormControl('', [Validators.pattern(/^[A-Z][a-z]{2,10}/),
    Validators.required ]),
  prenom: new FormControl('', [ Validators.required,
    this.checkPrenomValidator ]),
  adresse: new FormGroup({
    rue: new FormControl(''),
    ville: new FormControl(''),
    codePostal: new FormControl('')
  })
});
```

Reactive Form : FormBuilder

- Une classe service défini par Angular
 - **il faut l'injecter** dans le constructeur pour l'utiliser
 - **permet de simplifier la construction** d'un formulaire en évitant les répétitions

```
personneForm = new FormGroup({
  id: new FormControl('', Validators.required),
  nom: new FormControl('', [Validators.pattern(/^[A-Z][a-z]{2,10}/),
    Validators.required] ),
  prenom: new FormControl('', [ Validators.required,
    this.checkPrenomValidator ]),
  adresse: new FormGroup({
    rue: new FormControl(''),
    ville: new FormControl(''),
    codePostal: new FormControl('')
  })
});
constructor() { }
```

```
personneForm = this.fb.group({
  id: [null, Validators.required],
  nom: ['doe', [Validators.pattern(/^[A-Z][a-z]{2,10}/),
    Validators.required]],
  prenom: ['', [ Validators.required,
    this.checkPrenomValidator ]],
  adresse: this.fb.group({
    rue: [''],
    ville: [''],
    codePostal: ['']
  }),
});
constructor(private fb: FormBuilder) { }
```

Reactive Form : FormArray

- **FormArray** : Définit dans FormBuilder
 - Il permet de définir un tableau de taille indéterminée de FormControl

```
personneForm = this.fb.group({  
  id: [null, Validators.required],  
  nom: ['doe', [Validators.pattern(/^[A-Z][a-z]{2,10}/),  
    Validators.required]],  
  prenom: ['', [Validators.required,  
    this.checkPrenomValidator ]],  
  adresse: this.fb.group({  
    rue: ['',  
    ville: ['',  
    codePostal: ['']  
  }],  
  sports: this.fb.array([  
    this.fb.control('')  
  ])  
});
```

- Pour afficher instantanément les sports ajoutés, il faut retourner le FormArray

```
get sports() {  
  return this.personneForm.get('sports') as  
    FormArray;  
}
```

Reactive Form : FormArray

- Dans le Template il faut ajouter le **FormArray** au formulaire

```
<div formArrayName="sports">
  <h6>Sports </h6>
  <button (click)="ajouterSport()">
    Ajouter sport
  </button>
  <div *ngFor="let sport of sports.controls; let i=index">
    <div>
      Sport :
      <input type="text" [formControlName]="i">
    </div>
  </div>
</div>
```

Données Personnelles

Identifiant : 1

Nom :

Prénom :

Adresse

Rue :

Ville :

Code postal :

Sports

Ajouter sport

Sport : Running

Sport : Natation

Sport : Musculation

Envoyer

- Ajouter la méthode **ajouterSport()** dans le TypeScript du composant

```
ajouterSport() {
  this.sports.push(this.fb.control(''));
}
```

Reactive Form : Exercice

#8

Entant que utilisateur

authentifié,

je veux ajouter un nouveau

contact

Edit Contact Reactive FORM Component

Add Contact

Name

Sergey Brin

Email

sergey.brin@gmail.com

Website

http://google.com

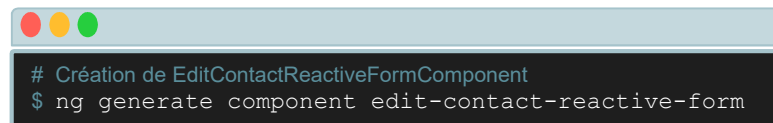
Projects (separated with comma",")

Google, Alphabet Inc

Save

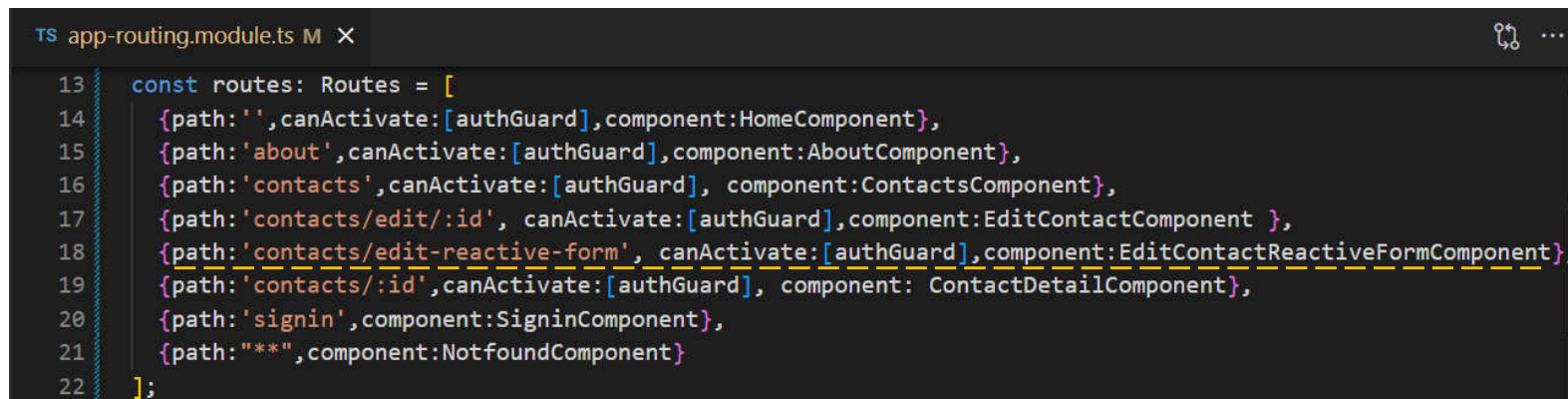
Reactive Form : Ajouter un contact

- **Etape 1:** Créer le component EditContactReactiveFormComponent



```
# Création de EditContactReactiveFormComponent
$ ng generate component edit-contact-reactive-form
```

- **Etape 2:** Ajouter la route /edit-reactive-form au AppRoutingModuleModule



```
TS app-routing.module.ts M X
13  const routes: Routes = [
14    {path: '', canActivate: [authGuard], component: HomeComponent},
15    {path: 'about', canActivate: [authGuard], component: AboutComponent},
16    {path: 'contacts', canActivate: [authGuard], component: ContactsComponent},
17    {path: 'contacts/edit/:id', canActivate: [authGuard], component: EditContactComponent },
18    {path: 'contacts/edit-reactive-form', canActivate: [authGuard], component: EditContactReactiveFormComponent},
19    {path: 'contacts/:id', canActivate: [authGuard], component: ContactDetailComponent},
20    {path: 'signin', component: SigninComponent},
21    {path: '**', component: NotFoundComponent}
22  ];
```

Reactive Form : Ajouter un contact

- **Etape 3:** Ajouter le bouton Add contact with reactive form à ContactsComponent

Contacts Component

Contact list

Name		
Doug Cutting	<button>Detail</button>	<button>Delete</button>
Bill Gates	<button>Detail</button>	<button>Delete</button>
Mark Zuckerberg	<button>Detail</button>	<button>Delete</button>

```
contacts.component.html X
src > app > contacts > contacts.component.html > div.card.border-primary.mb-3.mt-3 > div.card-b
2 | Add contact
3 | </button>
4 | <button class="btn btn-warning ml-3" (click)="onAddContactReactiveForm()">
5 |   Add contact with reactive form
6 | </button>
7 | <div class="card border-primary mb-3 mt-3">
```

```
TS contacts.component.ts X
src > app > contacts > TS contacts.component.ts > ...
27 | onAddContactReactiveForm(){
28 |   this.router.navigate(['contacts/edit-reactive-form']);
29 | }
```

- **Etape 4:** Importer le module **ReactiveFormsModule**

```
> app > TS app.module.ts > ...
  imports: [
    BrowserModule,
    AppRoutingModule,
    ReactiveFormsModule,
    FormsModule
  ],
```

Reactive Form : Ajouter un contact

- **Etape 5:** préparer le formulaire dans la classe du composant `edit-contact-reactive-form.component.ts`

```
TS edit-contact-reactive-form.component.ts X
src > app > edit-contact-reactive-form > TS edit-contact-reactive-form.component.ts > EditContactReactiveFormComponent
1 import { Component, OnInit } from '@angular/core';
2 import { FormBuilder, FormGroup } from '@angular/forms';
3 import { Router } from '@angular/router';
4
5 @Component({
6   selector: 'app-edit-contact-reactive-form',
7   templateUrl: './edit-contact-reactive-form.component.html',
8   styleUrls: ['./edit-contact-reactive-form.component.css']
9 })
10 export class EditContactReactiveFormComponent implements OnInit {
11   contactForm : FormGroup
12   constructor(private formBuilder: FormBuilder, private router: Router) { }
13
14   ngOnInit(): void {
15     this.initForm();
16   }
17   initForm(){
18     this.contactForm = this.formBuilder.group({
19       name : '',
20       email : '',
21       website : '',
22       projects : '',
23     })
24   }
}
```

On reviendra au traitement associé à la soumission du formulaire

```
TS edit-contact-reactive-form.component.ts X
src > app > edit-contact-reactive-form > TS edit-contact-reactive-form.component.ts > ...
26   onSubmit(){
27     //it's provisional
28     console.log(this.contactForm.value);
29   }
30   onContacts() {
31     this.router.navigate(['/contacts']);
32   }
}
```

Reactive Form : Ajouter un contact

- Etape 6: préparer le template `edit-contact-reactive-form.component.html`

```
edit-contact-reactive-form.component.html X
src > app > edit-contact-reactive-form > edit-contact-reactive-form.component.html > div.card.border-primary.mb-3 > div.card-body > form > div
1 <div class="card border-primary mb-3">
2   <div class="card-header bg-primary text-white"> Edit Contact Component</div>
3   <div class="card-body">
4     ----- <h3>Add Contact with reactive form</h3> -----
5     <form (ngSubmit)="onSubmit()" [formGroup]="contactForm">
6       <div class="form-group">
7         <label for="name">Name</label>
8         <input type="text" id="name" class="form-control" formControlName="name" >
9       </div>
10      <div class="form-row">
11        <div class="col">
12          <label for="email">Email</label>
13          <input type="email" id="email" class="form-control" formControlName="email">
14        </div>
15        <div class="col">
16          <label for="website">Website</label>
17          <input type="url" id="website" class="form-control" formControlName="website">
18        </div>
19      </div>
20      <div class="form-group mt-2">
21        <label for="projects">Projects (separated with comma",")</label>
22        <input type="text" id="projects" class="form-control" name="projects" formControlName="projects">
23      </div>
24      <button type="submit" class="btn btn-primary">Save</button>
25    </form>
26  </div>
27 </div>
28 <button class="btn btn-primary" (click)="onContacts()">
29   Go to Contacts
30 </button>
```

Reactive Form : Résultat temporaire

localhost:4200/contacts/edit-reactive-form

Edit Contact Reactive FORM Component

Add Contact

Name

Sergey Brin

Email Website

sergey.brin@gmail.com http://google.com

Projects (separated with comma",")

Google, Alphabet Inc

Save

Go to Contacts

Elements Console Sources Network Performance Memory Application Security »

top Filter Default levels

```
edit-contact-reactive-form.component.ts:28
{name: "Sergey Brin", email: "sergey.brin@gmail.com", website: "http://google.com", projects: "Google, Alphabet Inc"}
```

Reactive Form : Exercice (Validation)

#4 BFN

Règles de validation :

- ✓ Les champs sont obligatoires
- ✓ Une adresse e-mail valide
- ✓ Une URL valide

Edit Contact Reactive FORM Component

Add Contact

Name

Please provide a name.

Email

Please provide email.

Website

Please provide a website.

Projects (separated with comma",")

Please provide projects.

Save

Validation : Formulaire

- **Etape 1** : Intégrer les règles de validation dans `edit-contact-reactive-form.component.ts` :

```
TS edit-contact-reactive-form.component.ts U X
18   intitForm():void{
19       this.contactForm=this.formBuilder.group(
20           { name:['',Validators.required],
21             email:['', [Validators.required,Validators.email]],
22             website:['',[Validators.required, Validators.pattern('https?://.+')]],
23             projects:['',Validators.required]
24         }
25     )
26 }
```

- **Etape 2** : Désactivation du bouton `save` si le formulaire est **invalide**

```
< edit-contact-reactive-form.component.html X
src > app > edit-contact-reactive-form > < edit-contact-reactive-form.component.html > div.card.border-primary.mb-3
25   <button type="submit" class="btn btn-primary" [disabled]="contactForm.invalid">Save</button>
26   </div>
```

Validation : Formulaire Ajouter un contact

- **Etape 3 :** Ajout des messages d'erreur (input name et input email)

```
edit-contact-reactive-form.component.html X
src > app > edit-contact-reactive-form > edit-contact-reactive-form.component.html > div.card.border-primary.mb-3 > div.card-body > form
4      <h3>Add Contact</h3>
5      <form (ngSubmit)="onSubmit()" [formGroup]="contactForm">
6        <div class="form-group">
7          <label for="name">Name</label>
8          <input type="text" id="name" class="form-control" formControlName="name"
9            [ngClass]="{'is-invalid':contactForm.get('name').dirty && contactForm.get('name').invalid}">
10         <div *ngIf="(contactForm.get('name').dirty && contactForm.get('name').invalid)" class="invalid-feedback">
11           <div *ngIf="contactForm.get('name').hasError('required')">
12             Please provide a name.
13           </div>
14         </div>
15       </div>
16       <div class="form-row">
17         <div class="col">
18           <label for="email">Email</label>
19           <input type="email" id="email" class="form-control" formControlName="email"
20             [ngClass]="{'is-invalid':contactForm.get('email').dirty && contactForm.get('email').invalid}">
21           <div *ngIf="(contactForm.get('email').dirty && contactForm.get('email').invalid)" class="invalid-feedb
22             <div *ngIf="contactForm.get('email').hasError('required')">
23               Please provide email.
24             </div>
25             <div *ngIf="contactForm.get('email').hasError('email')">
26               Please provide a valid email address.
27             </div>
28           </div>
29         </div>
```

©MAD Mohamed Amr Abdelhakem@fsb.ucar.tn

Validation : Formulaire Ajouter un contact

- **Etape 3** : Ajout des messages d'erreur (input website et input projects)

```
edit-contact-reactive-form.component.html X
src > app > edit-contact-reactive-form > edit-contact-reactive-form.component.html > div.card.border-primary.mb-3 > div.card-body > form

30      <div class="col">
31          <label for="website">Website</label>
32          <input type="url" id="website" class="form-control" formControlName="website"
33              [ngClass]="{'is-invalid':contactForm.get('website').dirty && contactForm.get('website').invalid}">
34          <div *ngIf="(contactForm.get('website').dirty && contactForm.get('website').invalid)" class="invalid-feedback">
35              <div *ngIf="contactForm.get('website').hasError('required')">
36                  Please provide a website.
37              </div>
38              <div *ngIf="contactForm.get('website').hasError('pattern')">
39                  Please provide a valid URL (http://...).
40              </div>
41          </div>
42      </div>
43  </div>
44  <div class="form-group mt-2">
45      <label for="projects">Projects (separated with comma",")</label>
46      <input type="text" id="projects" class="form-control" name="projects" formControlName="projects"
47          [ngClass]="{'is-invalid':contactForm.get('projects').dirty && contactForm.get('projects').invalid}">
48      <div *ngIf="(contactForm.get('projects').dirty && contactForm.get('projects').invalid)" class="invalid-feedback">
49          <div *ngIf="contactForm.get('projects').hasError('required')">
50              Please provide projects.
51          </div>
52      </div>
53  </div>
```

© Mohamed El Ghannouchi - m.ghannouchi@fsb.ucar.tn

Résultat

Two side-by-side screenshots of an "Add Contact" form, titled "Edit Contact Reactive FORM Component".

Left Screenshot (Generic Errors):

- Name:** Input field containing "Sergey Brin".
- Email:** Input field with error message: "Please provide email." (indicated by a red dashed arrow).
- Website:** Input field with error message: "Please provide a website." (indicated by a red dashed arrow).
- Projects (separated with comma ","):** Input field with error message: "Please provide projects." (indicated by a red dashed arrow).
- Save Button:** Highlighted with a yellow box.

Right Screenshot (Specific Errors):

- Name:** Empty input field with error message: "Please provide a name." (indicated by a red dashed arrow).
- Email:** Input field containing "se" with error message: "Please provide a valid email address." (indicated by a red dashed arrow).
- Website:** Input field containing "google.com" with error message: "Please provide a valid URL (http://...)." (indicated by a red dashed arrow).
- Projects (separated with comma ","):** Input field with error message: "Please provide projects." (indicated by a red dashed arrow).
- Save Button:** Highlighted with a yellow box.

Yellow dashed arrows point from the central text box to the specific error messages in both forms.

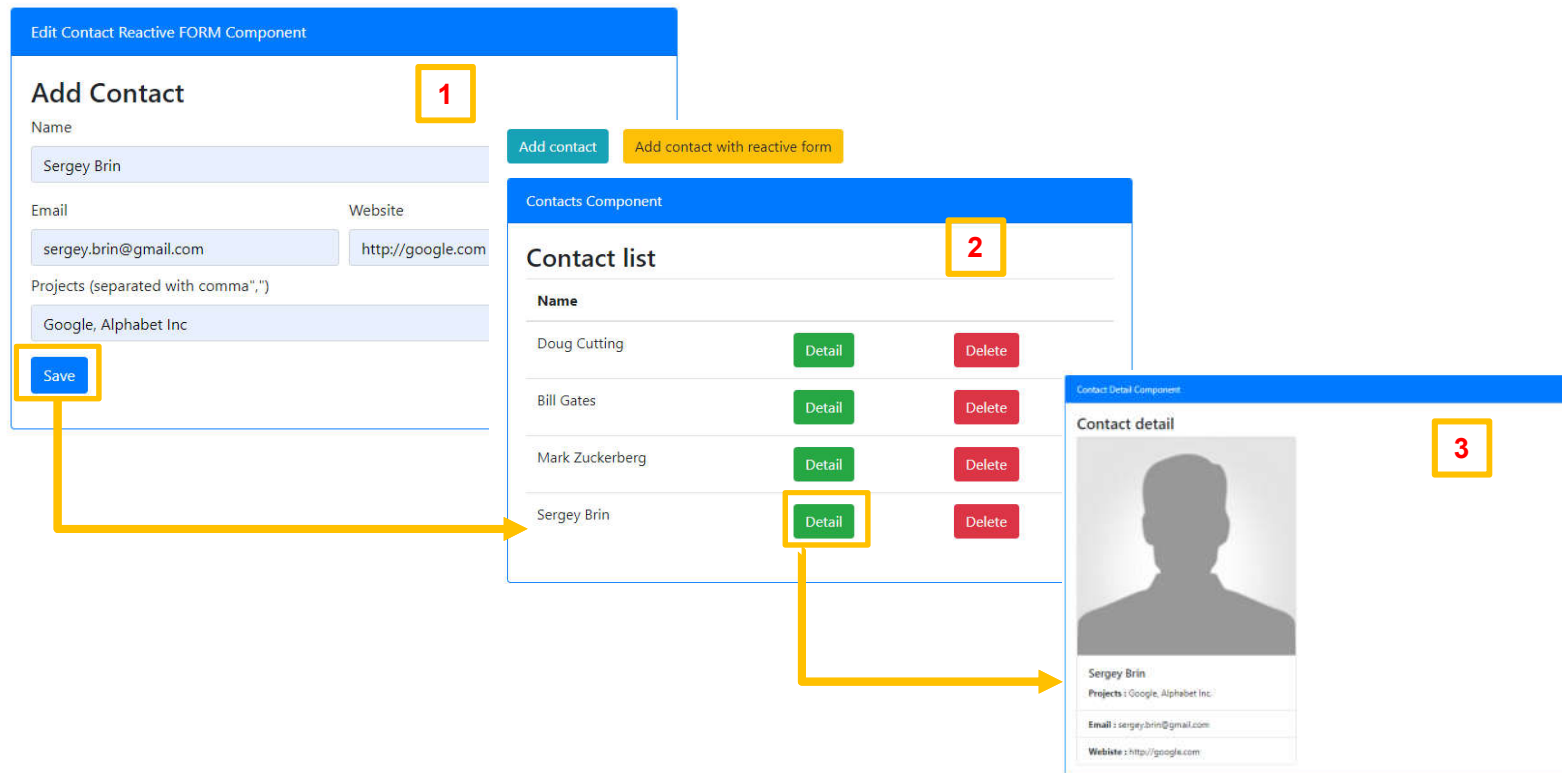
Chaque type d'erreur a un message

Terminer le traitement d'ajout d'un nouveau contact

```
edit-contact-reactive-form.component.ts X
rc > app > edit-contact-reactive-form > TS edit-contact-reactive-form.component.ts > EditContactReactiveFormComponent > onSubmit > contact
12 export class EditContactReactiveFormComponent implements OnInit {
13   contactForm : FormGroup
14   constructor(private FormBuilder: FormBuilder, private router: Router, private contactService : ContactService) { }
```

```
edit-contact-reactive-form.component.ts X
c > app > edit-contact-reactive-form > TS edit-contact-reactive-form.component.ts > ...
28
29 onSubmit(){
30   //create a new contact objetc and initialize it with the values of the form elemen
31   let contact : Contact = {
32     id : -1 ,
33     name : this.contactForm.get('name').value,
34     email : this.contactForm.get('name').value,
35     website : this.contactForm.get('name').value,
36     projects : [this.contactForm.get('name').value], //provide a FormArray
37     featured : false,
38     image : './assets/images/default-avatar.jpg' //default picture
39   };
40   this.contactService.addContact(contact);
41   this.router.navigate(['/contacts']) // redirect the user to contacts
42 }
```

Résultat final



03

Atelier 4

